

Documentation Technique – Projet HBnB

1. Introduction

1. Purpose of Documentation

The purpose of this documentation is to explain how my future clone of AirBnB (HBnB) will be structured and to describe the functioning of its hierarchical architecture. This document will be used to guide the future development coding,

2. Hbnb Project, Context.

This project has the goal of creating a simplified clone of AirBnB. By the end of this project, the main objectives are to implement several features on the website and to gain knowledge in using APIs and databases.

3. Scope of the Document.

In this document, I have included all types of diagrams and their descriptions (package diagram, class diagram, and four sequence diagrams). This document is intended to be used only for development and future implementation.

2. High Level Architecture

1. Architecture Overview

I think this project uses a layered architecture for several reasons: it is easy to manage, simple to update, more scalable, and easier to understand for a first project. The layered architecture is therefore the best option for our situation.

2. High-Level Package Diagram

1. Presentation layer

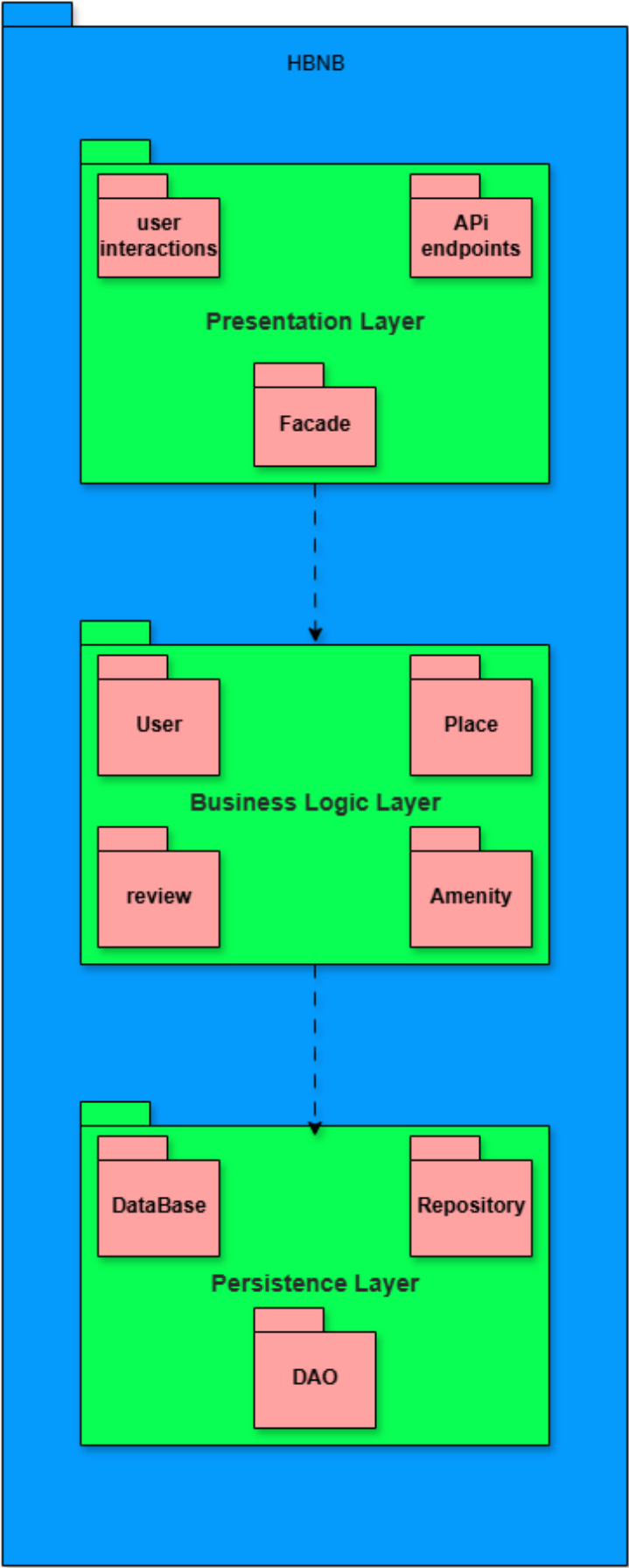
This layer has the function of linking the user interface with the business logic. Through this interface, the user can send HTTP requests via the API.

2. Business Logic Layer (Couche Logique Métier)

This layer is used to implement the core functions of the HBnB system. These functions are accessed through a facade, which adds an additional abstraction layer and hides the complexity of the business logic. This layer also handles requests to the database in order to store or retrieve data.

3. Persistence Layer (Couche Persistance des Données)

This layer has the goal of storing and deleting different types of data sent by the user. It also has the responsibility of searching the database and returning the results of user requests.



3. Business Logic Layer

1. Overview of the Business Logic Layer

The Business Logic Layer has the goal of receiving user requests, interpreting them, applying the necessary business rules, and forwarding the operations to the database. It also has the role of receiving the database's responses, filtering them, and sending the appropriate results back to the user.

2. Class Diagram (Business Logic Layer)

On this diagram, you can see the different links between the classes and how they interact with each other. In this diagram, the management class (Facade) serves as the entry point of the system. It manages the interactions and delegates tasks to the other classes.

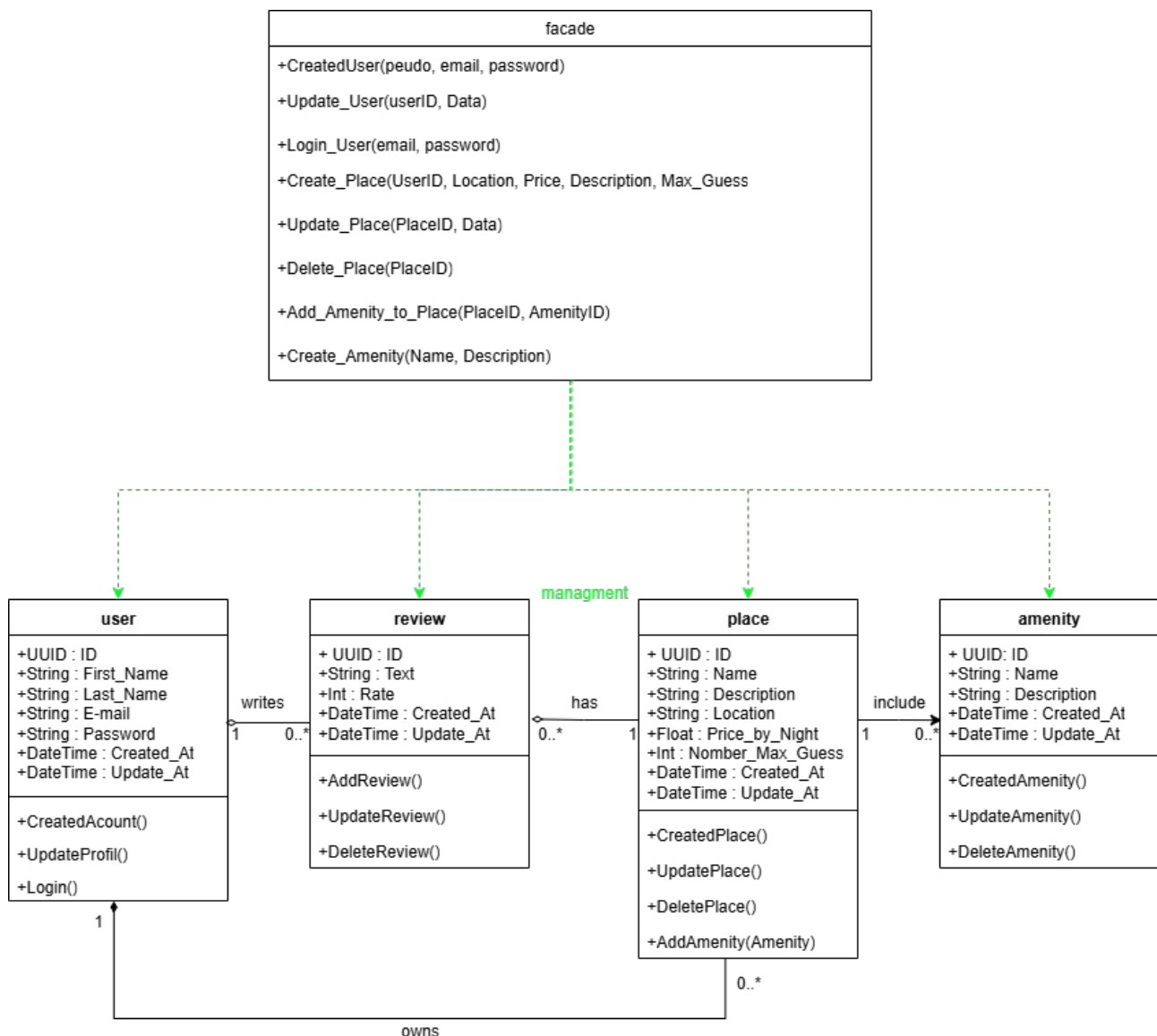
1. Legend of Diagram Notations

pointillé ---> : dependency : need but not posseded

full --->comon interactions

empty ---< : agregation : if delete can be existed

full ---< : composition : if delete is deleted



3. Description of Entities and Relationships

On this diagram, you can find several classes:

User: This class defines the main attributes and functionalities of a user. It represents the user's identity and allows access to the different features of the system.

Place: This class manages the different functions related to a house or apartment. It is linked to the User class in order to perform actions such as creating, modifying, or retrieving a place.

Review: This class manages the functionalities related to rating a place. It is linked to the User class to identify the author of the review.

Amenity: This class manages the services or features (e.g., Wi-Fi, parking, etc.) that an owner (User) can associate with different places.

Facade: This class manages all the other classes. It serves as the central entry point, handling requests and coordinating access to the different features of the system.

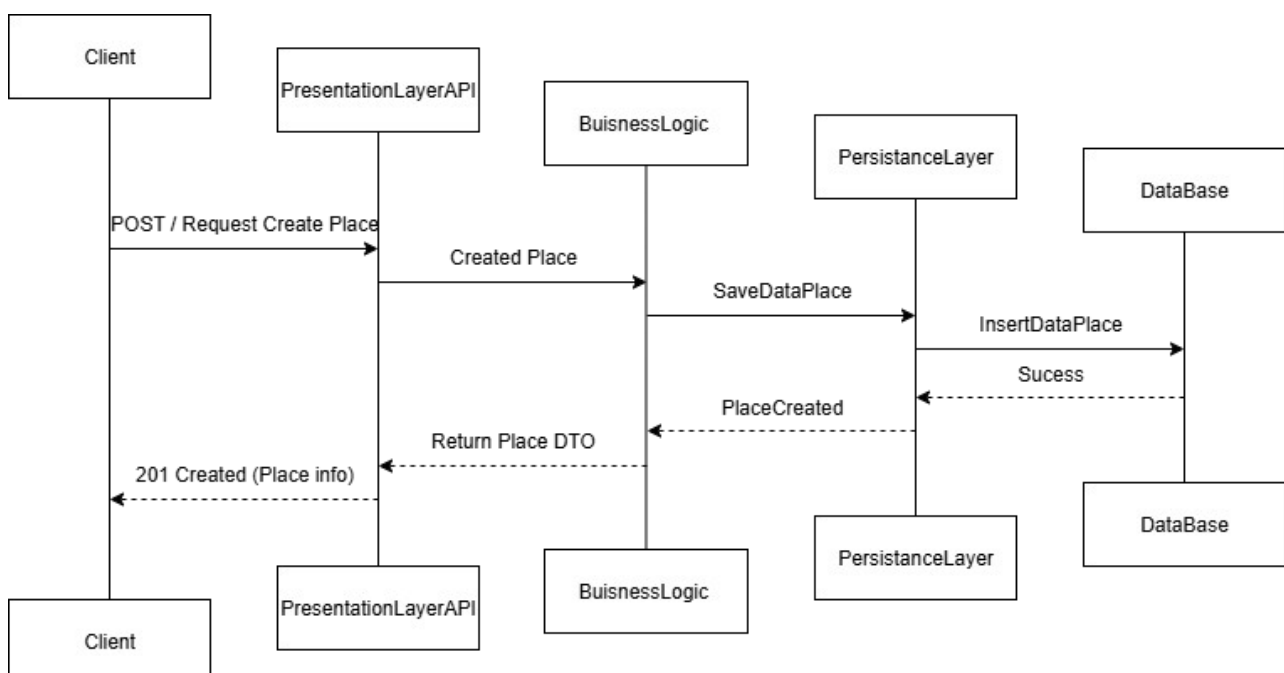
4. API Interaction Flow

1. Introduction to API Interactions

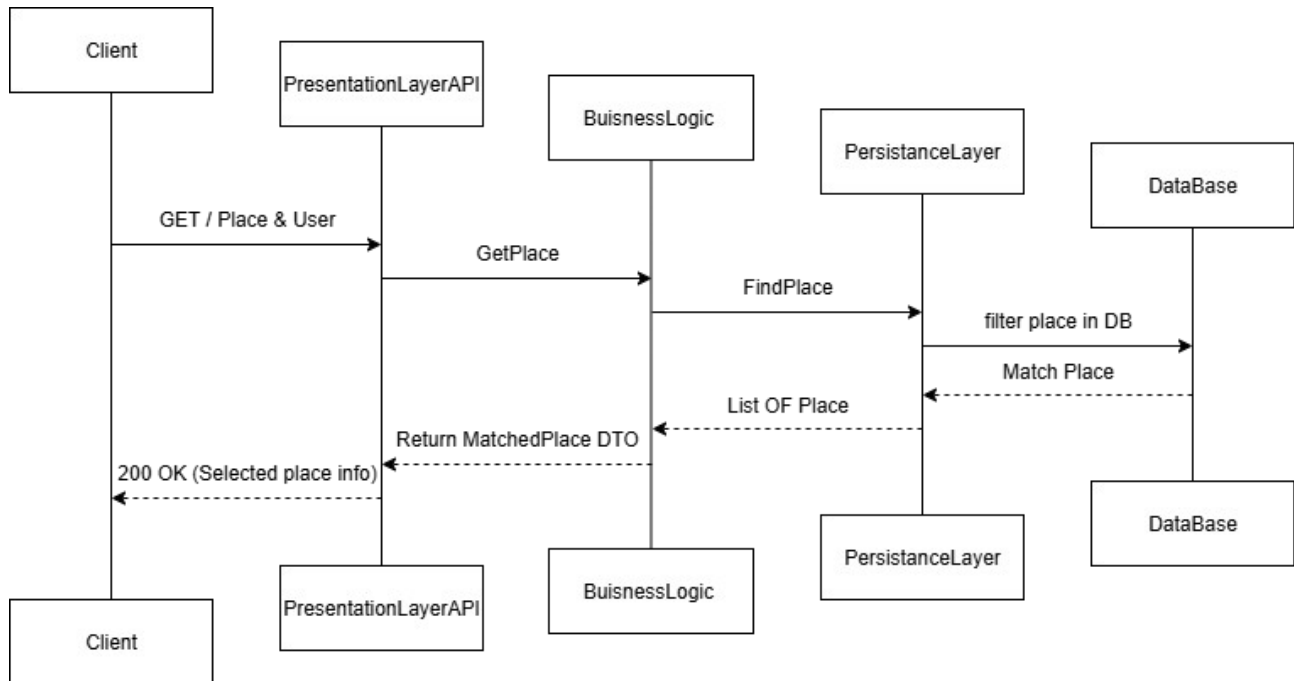
In my HBnB project, the API is used to transfer the data registered by the owner to the database, and to return the results of database queries back to the user. It acts as the main communication channel between the user interface and the business logic.

2. Sequence Diagrams (API Calls)

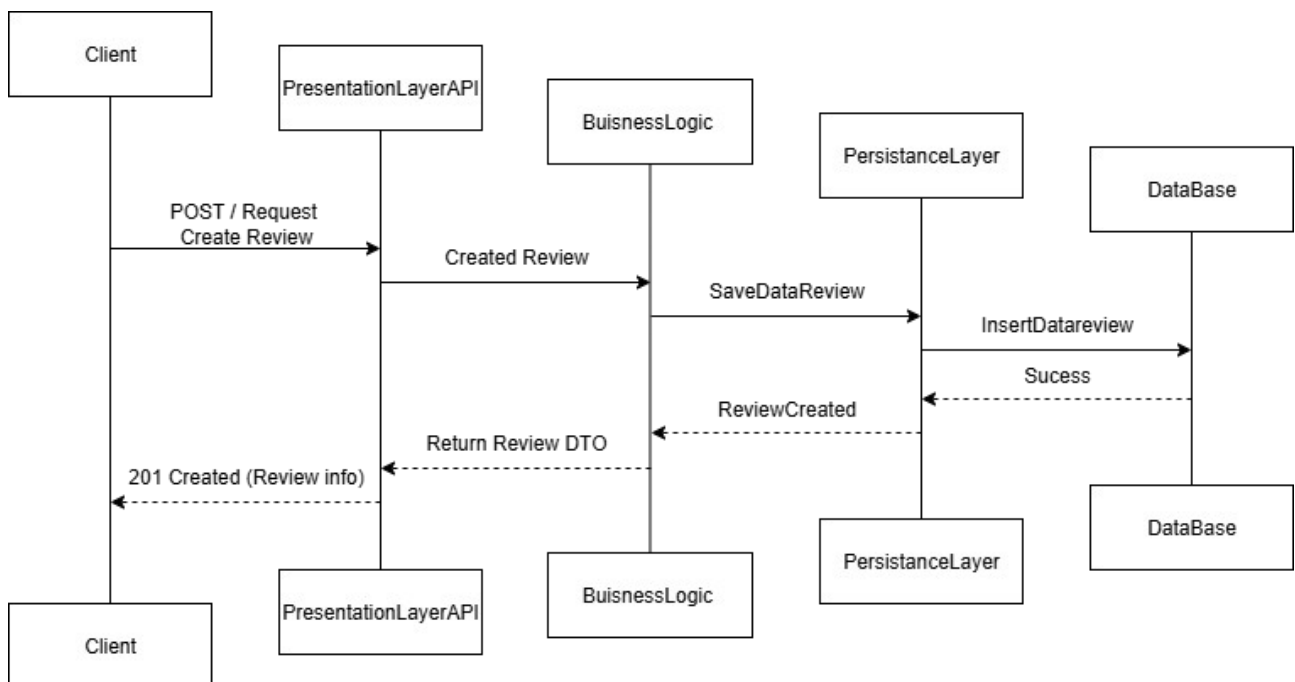
1. Sequence Diagram USER :



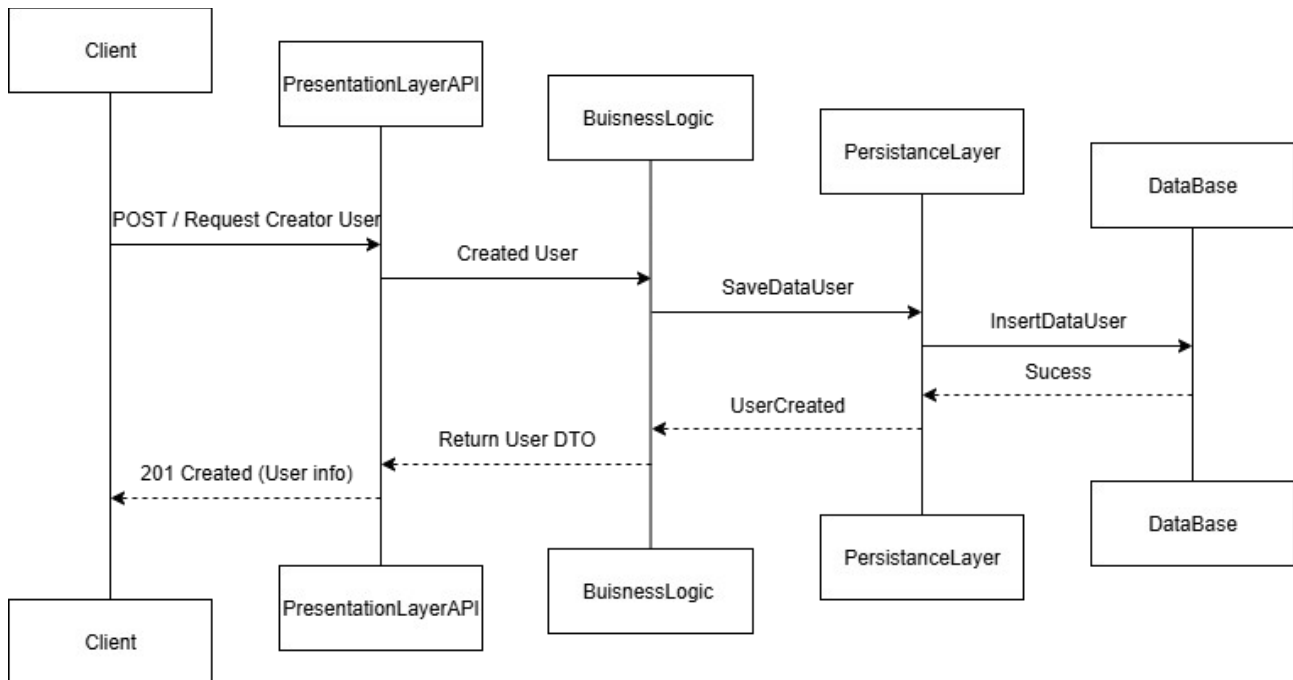
2. Sequence Diagram Place Creation



3. Sequence Diagram Review



4. Sequence Diagram Place Retrival



3. Explanation of Data Flow and Interactions

When a user makes a request, it can be of different types such as POST, GET, PUT, or DELETE. These requests are transferred to the API and then processed by the Business Logic layer, which determines the correct function to execute. If the request requires access to the database, the Persistence Layer communicates with it. Once the operation is completed (success or failure), the database response is returned through the layers to the user. The response is usually provided in a structured format such as JSON, which allows the client to clearly identify the result of the action.

5. Global Explanatory Notes

1. Consistency Across Diagrams

On this document, all diagrams use the same UML convention with same notation for class, package and interaction. For example, classes User, Place and Review always keep the same name and role, this avoid ambiguity.

The same terminology is used too for entities, methods and API call (like createUser, getPlaces). Relation on class diagram is the same on sequence diagram. Example: relation between User and Review is present on both diagrams.

This consistency make architecture more easy to understand and reduce risk of error on implementation.

2. Design Patterns Used

On this project we use different design patterns for improve structure and clarity of the system:

Facade Pattern: This pattern is used for centralize all requests from user. The facade class call other classes and hide complexity of business logic.

Layered Architecture Pattern: The project use layered structure (Presentation, Business Logic, Data) for separate responsibilities and make project more scalable.

3. Advantages of the Proposed Architecture

On this project, the proposed architecture gives several advantages:

Clarity and Organization: Each layer has a specific role (Presentation, Business Logic, Data), which makes the system easier to understand.

Maintainability: By separating responsibilities, modifications can be made in one part of the system without breaking the others.

Scalability: The layered structure allows the project to evolve and add new features without big changes.

Reusability: Some classes and functions can be reused in different contexts or extended for future developments.

Security and Control: The architecture makes it possible to filter and validate requests before sending them to the database.

6. Conclusion

This technical documentation has presented the global architecture of the HBnB project, including the high-level package diagram, the business logic class diagram, and the sequence diagrams for API interactions. Through these diagrams and explanations, we have defined the structure, the responsibilities of each layer, and the way components interact with each other. The chosen layered architecture ensures clarity, maintainability, and scalability, making it easier for developers to implement new features and extend the system in the future. By following this documentation, the development team has a clear blueprint to guide the implementation phases and ensure consistency across the project. Finally, this document is not only a reference for the current implementation but also a foundation for future improvements and evolutions of the HBnB system.

SORLI Thomas